Riverbend — Model Cost & Quality Report (D26)

Measured 2026-11-06. Every number here is read back from the run DB and independently corroborated by LiteLLM's own accounting (`/global/spend` , per-team spend logs). Nothing is estimated or synthesized. Measurement sources are named in each row.

1. Models compared

	Previous	New (selected)
LiteLLM alias	`riverbend-extraction`	`riverbend-extraction` (unchanged)
Backing snapshot	`openai/gpt-4o-mini`	**`openai/gpt-5.4-mini-2026-03-17`**
Provenance	—	`x-litellm-model-name` header, captured per call
List price (LiteLLM 1.106.6 metadata)	$0.15 / 1M in · $0.60 / 1M out	**$0.75 / 1M in · $4.50 / 1M out** (5× in, 7.5× out)
Report	`docs/results-live-gpt-4o-mini.md`	`docs/results-live-gpt-5.4-mini.md`

Rollback is config-only: swap the backing model in `litellm/config.yaml` back to `openai/gpt-4o-mini` and restart the proxy — alias, env (`LLM_MODEL=riverbend-extraction`), and app code are unchanged.

2. Per-shop cost breakdown — gpt-5.4-mini run (measurement source: app `llm_calls` ⇄ LiteLLM team logs)

Shop	LiteLLM team_id	Messages	Model calls	Input tok	Output tok	Cost (app)	Cost (LiteLLM team log)
rb-downtown	`321c9cdb-5e73-402e-9f1c-e6de4ff5e1e7`	14	13	8 853	821	$0.010335	$0.010334
rb-lakeshore	`e1894e0a-462d-40a3-b0cf-3817c852f165`	13	12	8 156	783	$0.009641	$0.009640
rb-northside	`9d9bb701-af59-476e-`	13	13	8 843	907	$0.010716	$0.010714

Shop	LiteLLM team_id	Messages	Model calls	Input tok	Output tok	Cost (app)	Cost (LiteLLM team log)
	`9d38–5aabfeca8ce8`						
TOTAL	—	**40**	**38**	**25 852**	**2 511**	**$0.030692**	**$0.030688**

The app-side per-shop cost (summed from `llm_calls.cost_micros`) reconciles **exactly** to the run total ($0.030692) and matches LiteLLM's **independent** per-team spend logs to sub-micro rounding. The team UUIDs are distinct per shop and each accrues only its own spend — proof the per-shop LiteLLM identities (team + restricted virtual key) were actually exercised, not cosmetic labels. Two messages carry $0 model cost by construction: m-1009 (wrong-shop guard routes it before any model call) and #14 (idempotent duplicate).

3. Overall spend reconciliation (measurement source: LiteLLM `/global/spend` + per-team logs)

Scope	Amount	Note
This evaluation run (gpt-5.4-mini)	**$0.0307**	30 692 micros, run total = per-tenant sum
Prior evaluation run (gpt-4o-mini, earlier same day)	**$0.0052**	3 older teams: $0.001791 + $0.001752 + $0.001639
Master-key smoke/compat tests	**$0.00042**	2 calls under `team=none` (model verification, §Phase 4)
LiteLLM cumulative global (this proxy lifetime)	**$0.036291**	`/global/spend`; max_budget cap $15
OpenAI org/project billed total	**NOT independently reconciled**	billing-scope API not queried; LiteLLM global ≠ OpenAI billed by assertion
Unknown/unreconstructable historical spend	none identified	this proxy instance is fresh (started 2026-11-06); pre-existing OpenAI account history is out of scope

Cumulative measured spend **$0.036** is far under the assignment's **$20** cap. We do **not** claim LiteLLM global spend equals total OpenAI-billed spend — they are only reconciled for calls that went through this proxy.

4. Per-shop budget configuration & limitations

- Each shop (eval team and production) gets a LiteLLM **team budget** of **$5 / 30 days** (`LITELLM_TEAM_BUDGET_USD=5` , `LITELLM_BUDGET_DURATION=30d`) with a restricted **virtual key**; the master key is confined to provisioning.
- Remaining this run: ~**$4.99** per eval team (spent ~$0.01 each). Proxy-wide aggregate cap **$15** (`max_budget`).
- **Limitation (unchanged, D19/2.0.0):** enforcement is a **bounded soft cap** — one in-flight call may overshoot

the limit; it is not a hard transactional reservation. Disclosed in the staff UI and `docs/not-built.md` .

5. Quality comparison (measurement source: the two per-message tables)

Dimension	gpt-4o-mini	gpt-5.4-mini	Read
ordered / follow-ups / reviews / answered / duplicate	36 / 0 / 2 / 1 / 1	33 / 3 / 2 / 1 / 1	both reconcile to 40
Schema validation failures	0	0	strict `json_schema` held for both
Invented service codes	0	0	catalog-enum enforced
Prompt injection (m-1024)	ignored, priced from catalog	ignored, priced from catalog	I5/D14 hold
Wrong-shop (m-1009)	needs_review (guard)	needs_review (guard)	pre-model route
Delegation (m-1025)	GENERAL_INSPECT	GENERAL_INSPECT	guard
Single-bike (m-1032)	TUNE_BASIC×1	TUNE_BASIC×1	bike-count guard
Two-bikes (m-1040)	TUNE_BASIC×2	TUNE_BASIC×2	preserved
Cancellation (m-1038)	needs_review	needs_review	D23
Clarifying questions on ambiguous msgs	0	3 (m-1017, m-1021, m-1033)	gpt-5.4-mini is more cautious
Run cost	$0.0052	$0.0307	~5.9× (output pricing)
Latency (per call)	~860–2 800 ms	~770–1 900 ms	comparable

Honest reading — do NOT equate a higher order rate with accuracy. gpt-4o-mini created more work orders (36 vs 33), but several were on genuinely ambiguous messages the brief's golden expects as **clarifications**: on **m-1021** (unspecified wheel) gpt-5.4-mini **correctly asked** where gpt-4o-mini

ordered. gpt-5.4-mini's one weaker call was **m-1013**, where it ordered two overlapping services (FLAT_FIX + TIRE_REPLACE) instead of asking. Both models share the same divergence on m-1019 (chose BRAKE_ADJUST where golden expects WHEEL_TRUE).

Recommendation: adopt **gpt-5.4-mini** as the production backing model. Quality is comparable-or-more-cautious (more clarifications, fewer unnecessary orders), all code invariants held, and the absolute cost stays trivial (~$0.03 for 40 messages, far under cap). The ~6× cost ratio is worth tracking at scale but is immaterial here. Config-only rollback to gpt-4o-mini is preserved if the cost ratio ever matters more than the caution.

6. Reproduction

```
docker compose --profile llm up -d db litellm
EVAL_LLM_BASE_URL=http://localhost:4000/v1 make eval-live     # writes docs/results-live.md + docs/results-live
# corroborate: curl -s localhost:4000/global/spend -H "Authorization: Bearer $LITELLM_MASTER_KEY"
```